

L10 · Δομές Δεδομένων

Σωροί & Ξένα Σύνολα

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Διαδικοί σωροί (heaps), *heapify-up* / *heapify-down*, ουρά προτεραιότητας, *heapsort*, και η δομή ξένων συνόλων (*union-find*) με βάρη στα δέντρα και συμπίεση μονοπατιών.

1 Τι θα δούμε

Στο προηγούμενο μάθημα είπαμε ότι ο **Dijkstra** και ο **Prim** τρέχουν σε $O(m \log n)$ «με δυαδικό σωρό», και ο **Kruskal** χρειάζεται μια δομή που απαντάει «είναι αυτές οι δύο κορυφές στην ίδια συνεκτική συνιστώσα;». Είπαμε «θα το δούμε αργότερα». Αυτή είναι η διάλεξη του «αργότερα».

Δύο δομές δεδομένων:

1. **Σωρός (heap)** — υλοποιεί μια **ουρά προτεραιότητας**: «δώσε μου το μικρότερο στοιχείο» σε $O(\log n)$. Αυτό κάνει τον Dijkstra και τον Prim γρήγορους.
2. **Ξένα σύνολα (union-find)** — υλοποιεί «σε ποιο σύνολο ανήκει το x ;» και «ένωσε δύο σύνολα» σχεδόν σε σταθερό χρόνο. Αυτό κάνει τον Kruskal γρήγορο.

Κλειδί

Δεν είναι «θεωρία για τη θεωρία». Κάθε δομή εδώ υπάρχει επειδή ένας αλγόριθμος που ήδη ξέρεις την χρειάζεται. Διάβασε κάθε ενότητα ρωτώντας: «ποια ακριβώς πράξη επιταχύνει αυτό, και σε ποιον αλγόριθμο;»

2 Μέρος Α' — Σωροί (Heaps)

2.1 Το πρόβλημα που λύνουν

Φαντάσου ότι έχεις μια συλλογή στοιχείων με **κλειδιά** (αριθμούς) και θέλεις, ξανά και ξανά, να ρωτάς: «**ποιο είναι το μικρότερο;**» και μετά να το **βγάζεις** από τη συλλογή. Ακριβώς αυτό κάνει ο Dijkstra: σε κάθε βήμα βγάζει την ανεξερεύνητη κορυφή με το μικρότερο $\pi(v)$.

Δομή	FindMin	Insert / Delete
Μη ταξινομημένος πίνακας	$O(n)$ — σάρωσε όλα	$O(1)$
Ταξινομημένος πίνακας	$O(1)$ — είναι στην άκρη	$O(n)$ — μετατόπιση
Σωρός	$O(1)$	$O(\log n)$

Διαίσθηση

Διαισθητικά: ο σωρός είναι ο **συμβιβασμός**. Ο ταξινομημένος πίνακας κρατάει τα πάντα σε τέλεια σειρά — υπερβολικά πολλή δουλειά. Ο μη ταξινομημένος δεν κρατάει τίποτα — πληρώνεις στο ψάξιμο. Ο σωρός κρατάει **μόνο όση τάξη χρειάζεται** για να ξέρει πάντα ποιο είναι το μικρότερο: κάθε γονιός μικρότερος από τα παιδιά του. Τίποτα παραπάνω.

2.2 Ορισμός

Κλειδί

Σωρός (heap): ένα **ισοσταθμισμένο δυαδικό δέντρο** που ικανοποιεί την **ιδιότητα διάταξης σωρού**: για κάθε κορυφή v με γονέα w ισχύει $\text{key}(w) \leq \text{key}(v)$.

Δύο κομμάτια, ξεχωριστά:

- **«Ισοσταθμισμένο δυαδικό δέντρο»** — αφορά το **σχήμα**. Κάθε επίπεδο γεμίζει πλήρως πριν ξεκινήσει το επόμενο, και το τελευταίο γεμίζει από αριστερά. Αυτό εγγυάται ύψος $\lfloor \log_2 n \rfloor$ — εκεί κρύβεται το $O(\log n)$ κάθε πράξης.
- **«Ιδιότητα διάταξης σωρού»** — αφορά τα **κλειδιά**. Κάθε γονιός $\leq$ κάθε παιδί του.

Πρόσεξε τι **δεν** λέει η ιδιότητα: δεν λέει τίποτα για τη σχέση ανάμεσα σε **αδέρφια**, ούτε ανάμεσα σε κορυφές διαφορετικών κλάδων. Ένας σωρός είναι «χαλαρά» ταξινομημένος — και ακριβώς γι' αυτό συντηρείται φθηνά.

Κλειδί

Συνέπεια: η ρίζα έχει πάντα το **ελάχιστο** κλειδί. (Είναι γονιός — άμεσα ή έμμεσα — όλων.) Άρα $\text{FindMin} = O(1)$: κοίτα τη ρίζα.

2.3 Αναπαράσταση με πίνακα

Το ωραιότερο κόλπο: ένας σωρός **δεν χρειάζεται δείκτες**. Επειδή το σχήμα είναι αυστηρά ισοσταθμισμένο, αποθηκεύουμε τις κορυφές σε έναν πίνακα $H[1..n]$, επίπεδο-επίπεδο από αριστερά προς δεξιά. Τότε για κάθε θέση i :

$$\text{parent}(i) = \lfloor i/2 \rfloor, \quad \text{leftChild}(i) = 2i, \quad \text{rightChild}(i) = 2i + 1$$

Δηλαδή «να κατέβεις στο παιδί» = πολλαπλασίασε με 2· «να ανέβεις στον γονέα» = διαίρεσε με 2. Καθαρή αριθμητική, μηδέν δείκτες.

Παράδειγμα. Ένας σωρός με κλειδιά $H = [2, 5, 8, 9, 11, 14, 18]$: η ρίζα $H[1] = 2$ έχει παιδιά $H[2] = 5$ και $H[3] = 8$ · το $H[2] = 5$ έχει παιδιά $H[4] = 9$ και $H[5] = 11$. Κάθε γονιός είναι μικρότερος από τα παιδιά του ($2 \leq 5, 8 \leq 9, 11 \leq 14, 18$), αλλά τα αδέρφια δεν είναι ταξινομημένα μεταξύ τους — και δεν χρειάζεται.

2.4 Heapify-up — επιδιόρθωση προς τη ρίζα

Όταν προσθέτουμε ένα στοιχείο, το βάζουμε στο **τέλος** του πίνακα (πρώτη ελεύθερη θέση — διατηρεί το σχήμα). Όμως το κλειδί του μπορεί να είναι **πολύ μικρό** και να παραβιάζει την ιδιότητα: μπορεί να είναι μικρότερο από τον γονέα του. Η **Heapify-up** το διορθώνει **σπρώχνοντάς το προς τα πάνω**:

```

Heapify-up(H, i):
  Εάν i > 1 τότε:
    j ← parent(i) = ⌊i/2⌋
    Εάν key(H[i]) < key(H[j]) τότε:
      Εναλλαγή των H[i] και H[j]
      Heapify-up(H, j)

```

Διαίσθηση

Διαισθητικά: το νέο, πολύ ελαφρύ στοιχείο «αναδύεται» σαν φυσαλίδα προς την επιφάνεια. Σε κάθε βήμα ανταλλάσσεται με τον γονέα του, μέχρι ή να βρει γονέα μικρότερο από αυτό, ή να γίνει το ίδιο ρίζα.

Γιατί διορθώνει σωστά: όταν ανταλλάσσουμε το $H[i]$ με τον γονέα του, το μικρό κλειδί ανεβαίνει — και ο γονέας, που ήταν μεγαλύτερος, κατεβαίνει σε μια θέση όπου τα παιδιά του ήταν ήδη $\geq$ από το παλιό περιεχόμενό της, άρα $\geq$ από αυτόν. Δεν δημιουργείται νέα παραβίαση πουθενά.

Χρόνος: το μονοπάτι προς τη ρίζα έχει μήκος $\leq \log_2 n$, και κάθε βήμα είναι $O(1)$. Άρα $\text{Heapify-up} = O(\log n)$.

Χρήση — Insert: τοποθέτησε το νέο στοιχείο στο τέλος, κάλεσε Heapify-up . Συνολικά $O(\log n)$.

2.5 Heapify-down — επιδιόρθωση προς τα φύλλα

Συμμετρικό πρόβλημα: μια κορυφή έχει κλειδί **πολύ μεγάλο**, μεγαλύτερο από κάποιο παιδί της. Τη σπρώχνουμε **προς τα κάτω**:

```

Heapify-down(H, i):
  Έστω l = 2i, r = 2i + 1 (παιδιά της i)
  Έστω j η θέση του παιδιού με το ΜΙΚΡΟΤΕΡΟ κλειδί (αν υπάρχουν παιδιά)
  Εάν υπάρχει j και key(H[j]) < key(H[i]) τότε:
    Εναλλαγή των H[i] και H[j]
    Heapify-down(H, j)

```

Προσοχή

Κρίσιμη λεπτομέρεια: ανταλλάσσουμε με το **μικρότερο** από τα δύο παιδιά — όχι με όποιο τύχει. Αν ανταλλάσσαμε με το μεγαλύτερο, αυτό θα κατέληγε γονιός του μικρότερου αδελφού του και θα παραβιαζόταν ξανά η ιδιότητα. Με το μικρότερο, ο νέος γονιός είναι σίγουρα $\leq$ και από τα δύο παιδιά.

Χρόνος: ίδιο επιχείρημα — μονοπάτι προς τα φύλλα μήκους $\leq \log_2 n$. $\text{Heapify-down} = O(\log n)$.

2.6 Ουρά προτεραιότητας

Συνδυάζοντας τα παραπάνω παίρνουμε την **ουρά προτεραιότητας** — μια δομή που πάντα ξέρει «τι έχει σειρά»:

Πράξη	Τι κάνει	Χρόνος
CreateHeap(H)	κενός σωρός για $\leq n$ στοιχεία	$O(n)$
Insert(H, v)	βάλει στο τέλος, Heapify-up	$O(\log n)$
FindMin(H)	επίστρεψε τη ρίζα $H[1]$	$O(1)$
Delete(H, i)	βγάλει τη θέση i , Heapify-down (ή up)	$O(\log n)$
ExtractMin(H)	FindMin + Delete της ρίζας	$O(\log n)$

Διαίσθηση

Πώς δουλεύει το ExtractMin: η απάντηση είναι η ρίζα — εύκολο. Αλλά τώρα η ρίζα είναι «τρύπα». Φέρνουμε το **τελευταίο** στοιχείο του πίνακα στη ρίζα (διατηρεί το σχήμα), και κάνουμε Heapify-down — αυτό το (μάλλον μεγάλο) στοιχείο βυθίζεται στη σωστή του θέση.

Η ουρά προτεραιότητας υποστηρίζει επίσης decreaseKey(H, v, k): μειώνει το κλειδί ενός στοιχείου και καλεί Heapify-up. Αυτή ακριβώς είναι η πράξη που κάνουν ο Dijkstra και ο Prim όταν βρίσκουν φθηνότερο μονοπάτι/ακμή προς μια κορυφή.

2.7 Εφαρμογή — Heapsort

Με την ουρά προτεραιότητας παίρνουμε δωρεάν έναν αλγόριθμο ταξινόμησης:

1. CreateHeap και βάλει μέσα και τα n στοιχεία.
2. Κάλεσε ExtractMin n φορές — βγαίνουν σε **αύξουσα** σειρά.

Χρόνος: n φορές το $O(\log n) \rightarrow O(n \log n)$. Είναι το ίδιο φράγμα με τη συγχωνευτική ταξινόμηση του τρίτου μαθήματος, αλλά με εντελώς διαφορετική ιδέα.

2.8 Πώς οι σωροί επιταχύνουν τον Dijkstra και τον Prim

Τώρα κλείνει η υπόσχεση του προηγούμενου μαθήματος. Και οι δύο αλγόριθμοι κρατάνε μια ουρά προτεραιότητας από ανεξερεύνητες κορυφές:

- ExtractMin — διάλεξε την επόμενη κορυφή προς εξερεύνηση. Καλείται n φορές $\rightarrow O(n \log n)$.
- decreaseKey — όταν μια κορυφή v μπει στο S , για κάθε ακμή (v, w) ενημέρωσε την προτεραιότητα του w . Συνολικά μέχρι m φορές $\rightarrow O(m \log n)$.

Σύνολο: $O(m \log n)$. Χωρίς σωρό, το ExtractMin θα ήταν γραμμική σάρωση $O(n)$ και θα κατέληγες σε $O(n^2)$.

3 Μέρος Β' — Ξένα Σύνολα (Union-Find)

3.1 Το πρόβλημα που λύνουν

Θυμήσου τον **Kruskal** από το προηγούμενο μάθημα: σαρώνει ακμές σε αύξουσα σειρά κόστους και προσθέτει την $\{u, v\}$ **εκτός αν δημιουργεί κύκλο**. Πώς ελέγχει «δημιουργεί κύκλο;» Μια ακμή $\{u, v\}$ κλείνει κύκλο **ακριβώς όταν** οι u και v είναι **ήδη** στην ίδια συνεκτική συνιστώσα του μέχρι τώρα δέντρου.

Άρα χρειαζόμαστε μια δομή που κρατάει μια **διαμέριση** — μια συλλογή από **ανά δύο ξένα σύνολα** — και απαντά γρήγορα: «το u και το v είναι στο ίδιο σύνολο;»

3.2 Οι τρεις πράξεις

Κλειδί

Η δομή **ξένων συνόλων (union-find)** διατηρεί μια συλλογή $\mathcal{S} = \{S_1, \dots, S_k\}$ από ανά δύο ξένα σύνολα. Κάθε σύνολο έχει έναν **αντιπρόσωπο** (ένα μέλος του). Πράξεις:

- $\text{MakeSet}(x)$ — φτιάξε νέο σύνολο με μοναδικό μέλος (και αντιπρόσωπο) το x .
- $\text{Union}(x, y)$ — συγχώνευσε τα σύνολα που περιέχουν τα x και y σε ένα.
- $\text{Find}(x)$ — επέστρεψε τον αντιπρόσωπο του συνόλου που περιέχει το x .

Ο αντιπρόσωπος είναι το «όνομα» του συνόλου: δύο στοιχεία είναι στο ίδιο σύνολο **αν και μόνο αν** έχουν τον ίδιο Find .

3.3 Εφαρμογή — συνεκτικές συνιστώσες

Με αυτές τις τρεις πράξεις, η εύρεση των συνεκτικών συνιστωσών ενός γραφήματος γίνεται μηχανικά:

Συνεκτικές Συνιστώσες ($G = (V, E)$):

Για κάθε κορυφή $v \in V$:

$\text{MakeSet}(v)$

Για κάθε ακμή $\{u, v\} \in E$:

 Εάν $\text{Find}(u) \neq \text{Find}(v)$:

$\text{Union}(u, v)$

Στην αρχή κάθε κορυφή είναι μόνη της. Κάθε ακμή «κολλάει» τα δύο της άκρα στην ίδια συνιστώσα. Στο τέλος, για δύο κορυφές x, y , ισχύει «ίδια συνιστώσα» αν και μόνο αν $\text{Find}(x) = \text{Find}(y)$.

3.4 Μια αφελής προσέγγιση

Η πιο απλή ιδέα: ένας πίνακας $\text{label}[]$ όπου $\text{label}[x]$ = το «όνομα» του συνόλου του x .

- $\text{Find}(x)$ — επέστρεψε $\text{label}[x]$. $O(1)$ — εξαιρετικό.
- $\text{Union}(x, y)$ — διάλεξε ένα όνομα και **σάρωσε όλον τον πίνακα** αλλάζοντας κάθε εμφάνιση του άλλου ονόματος. $O(n)$ — απαίσιο.

Με n ενώσεις φτάνουμε $O(n^2)$. Πληρώνουμε ακριβά στο Union . Χρειαζόμαστε καλύτερη ισορροπία.

3.5 Κατευθυνόμενα δέντρα με ρίζα

Η σωστή ιδέα: αναπαριστούμε **κάθε σύνολο σαν ένα κατευθυνόμενο δέντρο με ρίζα**.

- Κάθε κορυφή έχει **έναν** δείκτη: προς τον **γονέα** της.
- Η **ρίζα** δείχνει στον εαυτό της (θηλιά) — και είναι ο **αντιπρόσωπος** του συνόλου.
- Πολλά ξένα σύνολα $\rightarrow$ ένα κατευθυνόμενο **δάσος** με ρίζες.

3.6 Υλοποίηση των πράξεων

- $\text{MakeSet}(x)$ — φτιάξε δέντρο με μία κορυφή x (θηλιά στον εαυτό της). $O(1)$.

- $\text{Find}(x)$ — ακολούθησε τους δείκτες-γονείς από το x μέχρι να φτάσεις στη ρίζα (την κορυφή με θηλιά). Επίστρεψε την.
- $\text{Union}(x, y)$ — βρες τις δύο ρίζες· κάνε τη ρίζα του ενός δέντρου να δείχνει στη ρίζα του άλλου (αφαιρώντας τη θηλιά της). $O(1)$ αφού έχεις τις ρίζες.

Προσοχή

Το πρόβλημα: το κόστος είναι ολόκληρο στο Find . Αν ένα δέντρο εκφυλιστεί σε **μονοπάτι** (αλυσίδα), το Find από το φύλλο διασχίζει $O(n)$ κορυφές. Πίσω στο τετράγωνο. Χρειαζόμαστε τα δέντρα να μένουν **κοντά**.

3.7 Βάρη στα δέντρα (union by size)

Πρώτη βελτίωση: για κάθε δέντρο αποθηκεύουμε το **μέγεθός** του, και στο $\text{Union}(x, y)$ κάνουμε **τη ρίζα του μικρότερου δέντρου να δείχνει στη ρίζα του μεγαλύτερου** (όχι ανάποδα).

Κλειδί

Θεώρημα. Με union-by-size, σε ένα σύνολο με k στοιχεία κάθε στοιχείο απέχει το πολύ $\log_2 k$ βήματα από τη ρίζα του. Άρα $\text{Find} = O(\log n)$, και αρκούν $2 \log n$ βήματα για να αποφασίσουμε αν δύο στοιχεία είναι στο ίδιο σύνολο.

Απόδειξη (επαγωγή στο πλήθος ενώσεων). Ισχυρισμός: δέντρο με k κορυφές έχει βάθος $\leq \log_2 k$.

Βάση: δέντρο με 1 κορυφή έχει βάθος $0 = \log_2 1$. ✓

Επαγωγικό βήμα: ένα δέντρο με k κορυφές προέκυψε από Union δύο δέντρων με k_1 και k_2 κορυφές, $k_1 \leq k_2$, $k_1 + k_2 = k$. Βάλουμε το μικρό (k_1) κάτω από το μεγάλο (k_2). Το βάθος του νέου δέντρου είναι το πολύ:

$$\max \left\{ \underbrace{\log_2 k_2}_{\text{το μεγάλο μένει ίδιο}}, \underbrace{1 + \log_2 k_1}_{\text{το μικρό κατεβαίνει 1}} \right\}$$

Αφού $k_1 \leq k/2$, έχουμε $1 + \log_2 k_1 \leq 1 + \log_2 (k/2) = \log_2 k$. Και $\log_2 k_2 \leq \log_2 k$. Άρα το βάθος $\leq \log_2 k$. ■

Διαίσθηση

Γιατί δουλεύει: μια κορυφή «κατεβαίνει» ένα επίπεδο μόνο όταν το δέντρο της ενώνεται κάτω από ένα **τουλάχιστον ισομέγεθες**. Άρα κάθε φορά που κατεβαίνει, το σύνολό της **τουλάχιστον διπλασιάζεται**. Δεν μπορεί να διπλασιαστεί πάνω από $\log_2 n$ φορές $\rightarrow$ βάθος $\leq \log_2 n$.

3.8 Συμπίεση μονοπατιών (path compression)

Δεύτερη βελτίωση, σχεδόν δωρεάν. Κάθε φορά που τρέχουμε $\text{Find}(v)$ έχουμε ήδη διασχίσει όλο το μονοπάτι από το v μέχρι τη ρίζα x . Πριν επιστρέψουμε, **ξανασυνδέουμε κάθε κορυφή αυτού του μονοπατιού απευθείας στη ρίζα x** .

Διαίσθηση

Διαισθητικά: «αφού πλήρωσα ήδη να ανέβω αυτό το μονοπάτι, ας το ισιώσω για την επόμενη φορά.» Η επόμενη Find σε οποιαδήποτε από αυτές τις κορυφές θα είναι ένα βήμα. Μικραίνουμε το **ύψος** των δέντρων χωρίς να πειράζουμε τα μεγέθη/βάρη.

Με **union-by-size + path compression** μαζί, μια ακολουθία m πράξεων σε n στοιχεία τρέχει σε $O(m \cdot \alpha(n))$, όπου α είναι η **αντίστροφη συνάρτηση Ackermann** — μεγαλώνει τόσο αφάνταστα αργά που για κάθε n που θα συναντήσεις ποτέ στο σύμπαν, $\alpha(n) \leq 4$. Πρακτικά **σταθερά**.

3.9 Πώς το union-find επιταχώνει τον Kruskal

Κλείνει και η δεύτερη υπόσχεση του προηγούμενου μαθήματος. Ο Kruskal:

```
Kruskal(G, c):
  Ταξινόμησε τις ακμές κατά κόστος          ->  $O(m \log n)$ 
  Για κάθε  $u \in V$ : MakeSet(u)
  Για  $i = 1$  έως  $m$ , με  $\{u, v\} = e_i$ :
    Εάν Find(u)  $\neq$  Find(v):                  <- «κλείνει κύκλο;»
      πρόσθεσε  $e_i$  στο T
      Union(u, v)
```

- **Ταξινόμηση:** $O(m \log n)$.
- m ζεύγη Find + έως $n - 1$ Union: $O(m \cdot \alpha(n))$ — ουσιαστικά γραμμικό.
- **Σύνολο:** $O(m \log n)$ — η ταξινόμηση κυριαρχεί.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. **Σωρός** — ισοσταθμισμένο δυαδικό δέντρο με ιδιότητα διάταξης (γονιός $\leq$ παιδιά). Αποθηκεύεται σε πίνακα: $\text{parent}(i) = \lfloor i/2 \rfloor$, παιδιά $2i$ και $2i + 1$.
2. **Heapify-up / Heapify-down** — επιδιορθώνουν μία παραβίαση σε $O(\log n)$, σπρώχνοντας ένα στοιχείο προς τη ρίζα ή προς τα φύλλα (πάντα προς το **μικρότερο** παιδί).
3. **Ουρά προτεραιότητας** — FindMin $O(1)$, Insert/Delete/ExtractMin $O(\log n)$. Δίνει **Heapsort** $O(n \log n)$ και κάνει τον **Dijkstra & Prim** $O(m \log n)$.
4. **Union-Find** — MakeSet, Union, Find πάνω σε διαμέριση σε ξένα σύνολα. Κάθε σύνολο = κατευθυνόμενο δέντρο με ρίζα = αντιπρόσωπο.
5. **Union by size** κρατάει το βάθος $\leq \log n$. **path compression** το μικραίνει κι άλλο. Μαζί $\rightarrow O(\alpha(n))$ ανά πράξη, πρακτικά σταθερό.
6. Το union-find κάνει τον **Kruskal** $O(m \log n)$ — η ταξινόμηση των ακμών είναι πια το ακριβό κομμάτι.